

Human-like autonomy emerges from self-play and a pinch of human data

Daphne Cornelisse¹, Julian Hunt², Zixu Zhang³, Kevin Joseph²
Jaime Fernández Fisac³, Eugene Vinitsky¹

¹NYU Tandon School of Engineering ²NYU Courant ³Princeton University

April 28, 2026

Abstract

Self-play reinforcement learning has recently taken the wheel in autonomous driving. The approach substitutes expensive, large-scale human demonstration data with cheap, large-scale simulation. While cost-effective, agents trained through self-play often behave unpredictably around unseen policies and fail to coordinate with human drivers. Previous works attempt to mitigate this behavioral incompatibility through reward engineering, which is labor-intensive and brittle. In this work, we instead learn driving policies by training on over six years of self-play simulation and 30 minutes of human demonstrations. Our agents achieve zero-shot coordination with real human driving trajectories using $125,000\times$ less human data than imitation learning baselines, at $1/30$ the number of GPU hours. The solution space of unconstrained driving policies is large; this small pinch of human data is sufficient to act as a regularizer towards ones that fit the road. Taken together, the results suggest that the barrier to human-compatible agent training may be lower than implied by the field’s reliance on large demonstration datasets, and points toward a cheaper and more scalable recipe for training agents that are both effective and human-compatible.

Keywords: Zero-Shot Coordination, Self-play Reinforcement Learning, Imitation Learning, Autonomous Driving

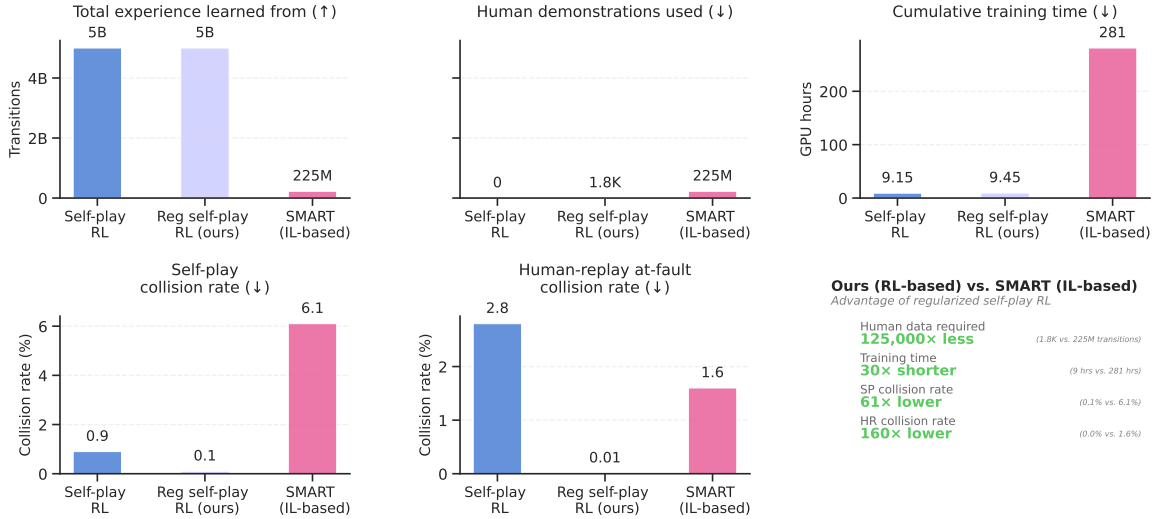


Figure 1: Comparison of approaches to human-compatible driving. **SMART-tiny-CLSFT** [48, 50] is a tokenized transformer (SMART architecture, Wu et al. [48]) trained by imitation learning on the full Waymo dataset and fine-tuned closed-loop with CAT-K [50]. **Regularized self-play RL** trains almost entirely on synthetic experience, using 30 minutes of human demonstrations as a behavioral anchor. This small anchor suffices for zero-shot coordination, at a fraction of the data and training time. **Self-play RL** uses the same minimal reward function but no human data, and subsequently fails to coordinate with human drivers.

1 Introduction

Self-play reinforcement learning (RL) is a core ingredient for training superhuman agents in strategic games [31, 40, 42] and, more recently, complex real-world domains such as autonomous driving [8, 10, 12, 23] and robotic manipulation [49]. The idea is quite elegant: an agent plays against a copy of itself, and as it improves, so does its opponent. This mechanism creates an automatically evolving curriculum [25] that brings the agent from random behavior to optimal behavior through entirely synthetic experience.

A central challenge remains. When a trained policy must interact with humans, it must align with human norms, expectations, and conventions, a requirement formalized as Zero-Shot Coordination (ZSC) [18]. Self-play RL offers no guarantee of this: agents may develop effective but alien strategies that are incompatible with human partners [1]. The higher the complexity and level of ambiguity of the game, the larger the solution space, and subsequently the higher the probability that the agent will converge to behavior that is effective but alien to humans.

Existing approaches address this in different ways. *Manual reward engineering* iteratively adds reward terms until the desired behavior emerges [12, 34], but is labor-intensive, domain-specific and brittle [24]. For instance, GIGAFLOW [12] requires nine individually tuned reward terms to produce naturalistic driving behavior. *Imitation learning* [5, 32, 33] trains directly on human demonstrations, but robustness requires broad behavioral coverage, so these approaches typically require large quantities of human demonstrations [3].

We take a different approach grounded in the following observation: synthetic experience has recently become very cheap, while human data has not. Modern RL frameworks and simulation infrastructure now generate 300K to 20 million steps per second on a single GPU [11, 43], giving self-play RL in simulation the ability to generate an endless sea of synthetic experience. Collecting human demonstrations, by contrast, requires manual effort and remains hard to scale. This suggests a natural role for human data: not as the primary source of training signal, but as a lightweight anchor that steers the policy away from effective yet behaviorally alien strategies. Regularizing self-play RL toward such an anchor has already shown promise in producing human-compatible agents in Diplomacy [2, 19] and driving [8, 9, 10, 46], but *how much* human data this actually requires has not been examined to date.

We present the first systematic study of this question. We regularize self-play PPO toward a behavioral cloning anchor trained on varying amounts of human demonstrations from the Waymo Open Motion Dataset [14, WOMD], and find that the answer is: *surprisingly little*. Human-compatible driving agents can be learned from as little as **30 minutes of human demonstrations**, roughly **0.04%** of the data used to train state-of-the-art imitation learning models, while using 1/30 of the GPU hours times faster. Performance follows a clear scaling law with anchor dataset size, and returns diminish rapidly beyond a modest threshold [QUANTIFY]. More broadly, the present results suggest that **wherever simulation is cheap, collecting a small amount of human demonstrations may suffice** to learn agents that are compatible with human play.

Our contributions are:

- We show that regularizing self-play RL toward a behavioral cloning anchor achieves zero-shot coordination with human drivers where pure self-play fails, without reward engineering and with 30 minutes of human demonstration data.
- We present the first systematic study of human data scaling under regularized self-play RL.
- We open-source the full codebase; each training run is reproducible in four hours on a single consumer-class GPU.

2 Related work

Imitation learning for autonomous driving. Imitation learning has been the dominant paradigm for training realistic traffic agents and driving policies (planners), leveraging large-scale datasets of real-world driving behavior [6, 14, 47]. A broad class of methods learns to forecast the future trajectories of agents in a scene, either marginally [16, 28, 35, 39, 44, 51] or jointly across multiple interacting agents [29, 54, 55]. A prominent line of work discretizes trajectories into tokens and trains autoregressive models to predict the next token [32, 38, 48], yielding diverse and realistic behavior at the cost of large training datasets. Diffusion models offer another route, learning distributions over

agent trajectories and environment features [22, 53] or modeling the joint behavior of multiple agents in a scene [20, 52]. These methods generate diverse and controllable traffic scenarios, but typically require extensive computation and large human datasets to achieve robust closed-loop performance [3]. Several recent works combine imitation with reinforcement learning to improve closed-loop robustness [26, 30, 50], but still rely on large demonstration datasets as the primary training signal. Our approach inverts this balance: synthetic self-play experience is used as the primary learning signal, and a small human dataset serves only as a behavioral anchor.

Self-play reinforcement learning in games. Self-play reinforcement learning has produced superhuman agents in games from Go and Chess [40, 41] to StarCraft II [45] and Stratego [31], all without human data. Superhuman play is not the same as human-compatible play, however. Many games admit multiple equilibria, and self-play need not converge to equilibria that are compatible with human partners [1, 18]. The failure has been shown in cooperative games such as Hanabi [4] and Diplomacy [1], where self-play agents develop internally consistent conventions that transfer poorly to human partners. The cause is reward underspecification: when the reward is defined as a score to maximize, there are often many ways to achieve it. In other words, the solution space is large. Previous work attempts to resolve this by sharing the reward by hand [12, 34]. For instance, GIGAFLow [12] demonstrates that reward engineering and domain randomization can produce naturalistic behavior at scale, at the cost of nine individually tuned reward terms. We avoid reward engineering entirely. A small amount of human data serves as a behavioral anchor, and self-play does the rest. This reduces a labor-intensive design problem to a one-hour data collection.

Human-regularized self-play reinforcement learning and search. One alternative to reward engineering is to regularize self-play toward a human anchor policy. This idea has been explored in Diplomacy, where KL regularization toward a human prior during both search and learning produced agents that coordinate more effectively with human partners [2, 19]. Jacob et al. [21] study KL-regularized search more broadly and show that it recovers human-like play across several games. In autonomous driving, Cornelisse and Vinitisky [9] show that regularizing PPO toward a behavioral cloning anchor produces more human-compatible driving agents on a small scale. More recently, Chang et al. [8] show that KL-regularized self-play can train human-like driving policies from scratch using SMART [48] as the behavioral anchor, a large tokenized model trained on the full 500,000-scenario Waymo dataset. [46] show that regularized self-play RL effectively adapts policies to new cities, without using human demonstrations from that city. In sum, the effectiveness of regularized self-play RL has been shown. What remains unexamined is the scaling law that governs it: how performance depends on the amount and quality of the human anchor data. The question matters now in a way it did not before. Simulation throughput has grown by orders of magnitude in recent years, shifting the bottleneck from synthetic experience to human data.

3 Method

3.1 Evaluation and metrics

Evaluation The goal is to train agents that *blend in* with the human driver population. Since on-road deployment is out of scope, we approximate the human population with logged human trajectories and evaluate in two settings, illustrated in Figure 2. In *self-play*, all agents are controlled by the trained policy. In *human-replay*, only the self-driving car (SDC) is controlled by the policy, and all other agents follow their logged trajectories. A human-compatible agent should score well in both: it should reach its goal without collisions or off-road events, whether its co-players are copies of itself or human drivers it has never trained against. A known failure mode of self-play RL is that agents overfit to their own population, finding effective but alien conventions that do not transfer to humans [1]. A good regularizer closes this gap. We therefore focus much of our analysis on the difference between the two evaluations rather than on either one in isolation.

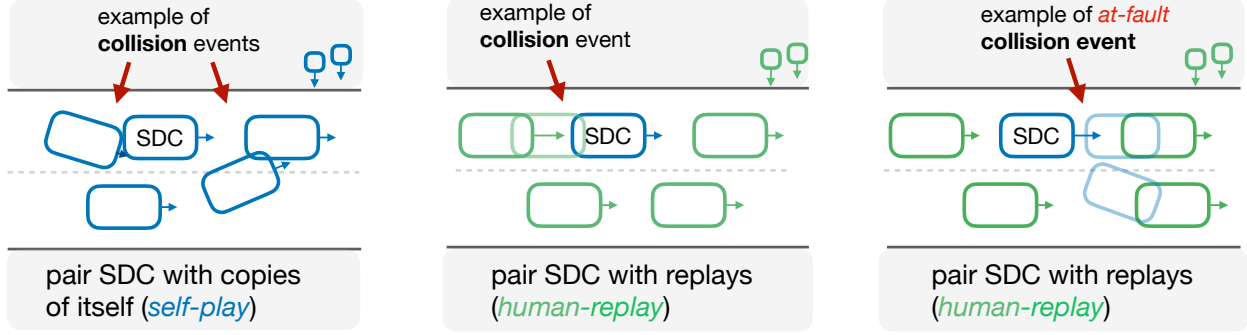


Figure 2: **Evaluation settings.** Self-play (left) and human-replay (center, right). Red arrows mark collisions. Rectangles are vehicles; squares are pedestrians. In human-replay, some collisions are effectively unavoidable: replay agents follow their logged trajectories and can drive into the controlled SDC from behind. We therefore distinguish between *collisions* (any contact) and *at-fault collisions* (contact caused by the controlled agent, following NAVSIM [13]).

Metrics and ceiling Our primary metric is the **score**: an agent scores 1 if it reaches its goal without colliding or going off-road, and 0 otherwise. To diagnose failure modes, we separately report **collision rate**, **at-fault collision rate**, **off-road rate**, and **route progress**. To quantify the transfer gap, we define the **ZSC gap**

$$\Delta_{\text{ZSC}} = \text{score}_{\text{self-play}} - \text{score}_{\text{human-replay}}, \quad (1)$$

where an ideal agent attains $\Delta_{\text{ZSC}} = 0$ with a perfect score in both settings. Score-based metrics capture whether agents complete their task safely, but not whether their behavior looks human. We therefore also report **distributional realism** using the Waymo Open Sim Agent Challenge (WOSAC) protocol [27], which compares statistics of simulated and logged trajectories across kinematic, interactive, and map-based features. All metrics are reported on held-out test scenarios unless stated otherwise; full definitions are given in Appendix D.2.

3.2 Simulation environment

We train policies $\pi(a_t^i \mid o_t^i)$ in PufferDrive 2.0 [11], a batched driving simulator that reaches 300k SPS on a single consumer-grade GPU. Environments are initialized from the Waymo Open Motion Dataset [14]: each 9-second scenario provides a roadgraph, a variable set of agents (cars, cyclists, pedestrians) up to $N = 32$, and per-agent initial poses and goals drawn from the logged trajectories. Each agent receives a partial, decentralized, ego-frame observation consisting of its own state, the $N - 1$ closest neighbors within 50 m, and up to 128 nearby road segments.

We avoid reward shaping and only use a sparse reward with high-level directions: +1 for reaching a goal position, −1 for collision or off-road, and 0 otherwise. This way, human-like behavior can emerge from regularization rather than from reward shaping. We truncate episodes when all agents have reached their destinations. Full details are summarized in Appendix A.

3.3 Regularized self-play reinforcement learning

Our method has two stages: we first train a behavioral cloning anchor on human demonstrations, then use it as a regularizer during self-play RL. Note that the RL policy is learned from scratch rather than fine-tuned from the anchor.

Step 1: Train the anchor policy To study how the amount of human demonstration data affects downstream performance, we train a set of anchor policies on nested subsets of the demonstrations. Given the full dataset $\mathcal{D} = \{(o_t^i, a_t^i)\}_{i=1}^{T \cdot K}$ of observation-action pairs from K scenarios of T timesteps each, we sample subsets $\mathcal{D}_n \subset \mathcal{D}$ corresponding to n scenarios, yielding roughly {10 min, 30 min, 3 h, 30 h} of human driving data. Each anchor τ_n is trained by minimizing the negative log-likelihood of the logged actions:

$$\tau_n = \arg \min_{\tau \in \mathcal{T}} \sum_{(o_t^i, a_t^i) \in \mathcal{D}_n} -\log \tau(a_t^i \mid o_t^i), \quad (2)$$

Figure 3 illustrates the data collection process in three scenarios. Since we have enough scenarios, we only use the SDC trajectory from each scenario, as it is typically the highest-quality trajectory in the scene. Each τ_n is then frozen and used as the regularizer during the subsequent self-play stage. See Appendix A.4 for the full details.

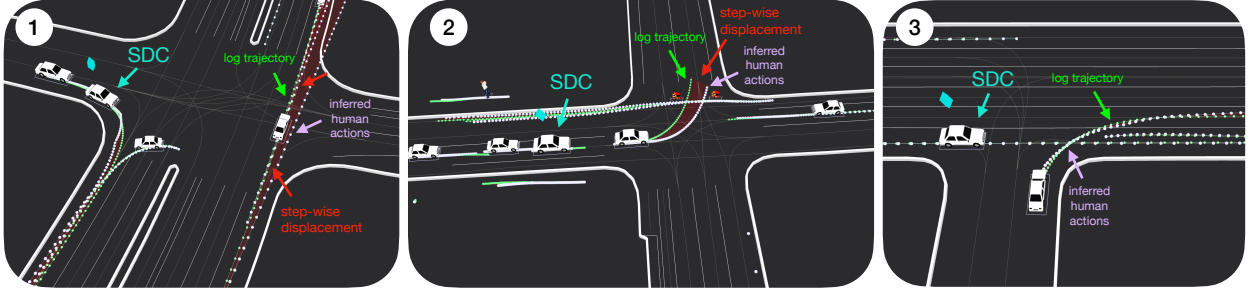


Figure 3: Three annotated example scenarios illustrating the human demonstration data collection process. The self-driving car (SDC), marked in cyan, is the Waymo vehicle whose human-driven trajectory we use as the demonstration. Logged trajectories are shown in green; purple trajectories show the result of stepping each agent through the simulator under the inferred delta-local actions. We select only the SDC trajectory because it is typically the cleanest demonstration in the scene; the visualized step-wise displacement illustrates a few low-quality (high-ADE) log trajectories that would otherwise contaminate the anchor. To support quick verification of the data quality, we add a new mode: puffer verify puffer_drive.

Step 2: Regularized self-play RL We train a new policy π_θ from scratch using PPO [37] in self-play, with each anchor τ_n acting as a behavioral regularizer. Concretely, we add a KL penalty from the anchor to the policy to the standard PPO objective:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{PPO}}(\theta) + \lambda \mathbb{E}_{o \sim \rho_{\pi_\theta}} [D_{\text{KL}}(\tau_n(\cdot | o) \parallel \pi_\theta(\cdot | o))], \quad (3)$$

where ρ_{π_θ} is the state distribution induced by the on-policy rollouts and $\lambda \geq 0$ controls the regularization strength. The KL term pulls π_θ toward the anchor’s action distribution on states the policy actually visits, rather than on the distribution of states in $\mathcal{D}_n$. Following [12, 19], we sample $\lambda \sim \text{Uniform}(0, 1)$ per episode and expose it as part of the observation, so that a single trained policy can be steered at inference toward more or less human-like behavior without retraining. Scenario metadata (Appendix A.1) is used to initialize the worlds during self-play (define the game). The full training objective and hyperparameters are given in Appendix B.

4 Experiments

This section summarizes the key results. Additional details and experiments are reported in the appendices, including details on datasets, network architectures, hyperparameters, and environment settings. The appendices also report ablations.

The sections are structured to answer the following questions:

1. *Scaling laws for human demonstration data:* How much human demonstration data is needed for strong performance in both the self-play and human-replay settings? How do these scaling laws compare to those of the state-of-the-art imitation learning baseline? (Section 4.1)
2. *The role of metadata and scenario diversity:* Driving datasets such as WOMD and NuPlan provide scenario metadata—road graphs and initial agent positions—that grounds simulation at a fraction of the cost of collecting human demonstrations. How does the number of training scenarios (maps) used for self-play influence agent performance? (Section 4.2)
3. *Behavioral analysis:* Human-likeness in driving is not easily captured by any single metric. We conduct a full behavioral analysis comparing regularized self-play agents against their unregularized counterparts and the imitation learning baseline. (Section 4.3)

4. *Correcting for bad anchor predictions in OOD states:* Behavioral cloning (BC) is widely used for distilling human demonstrations into policies because of its simplicity. But when a BC policy is deployed closed-loop, small prediction errors over time can bring the agent out-of-distribution states and do poorly. Regularized self-play agents use a BC anchor and inherit this weakness during training. The anchor policy may occasionally output garbage predictions when the policy rollouts are out-of-distribution states. Can self-play RL correct for this? (Section 4.4)

4.1 Scaling laws for human demonstrations

How much human data does regularized self-play need, and how does this compare to pure imitation learning? The second question matters because any apparent data efficiency on our side could simply reflect the homogeneity of the Waymo Open Dataset rather than a genuine property of the method.

To answer this, we benchmark against SMART-tiny-CLSFT [48, 50] (—●—), the state-of-the-art imitation-learning approach in this field. We train SMART on the same nested subsets of human demonstrations as regularized self-play (—●—). The two pipelines differ only in the second stage, where SMART-tiny-CLSFT applies CAT-K closed-loop fine-tuning [50] (see Appendix B.4 for full details). As a second baseline, we include plain self-play RL with a minimal reward function (Appendix B.2). Figure 4 shows the results visually; Table 1 reports the numbers.

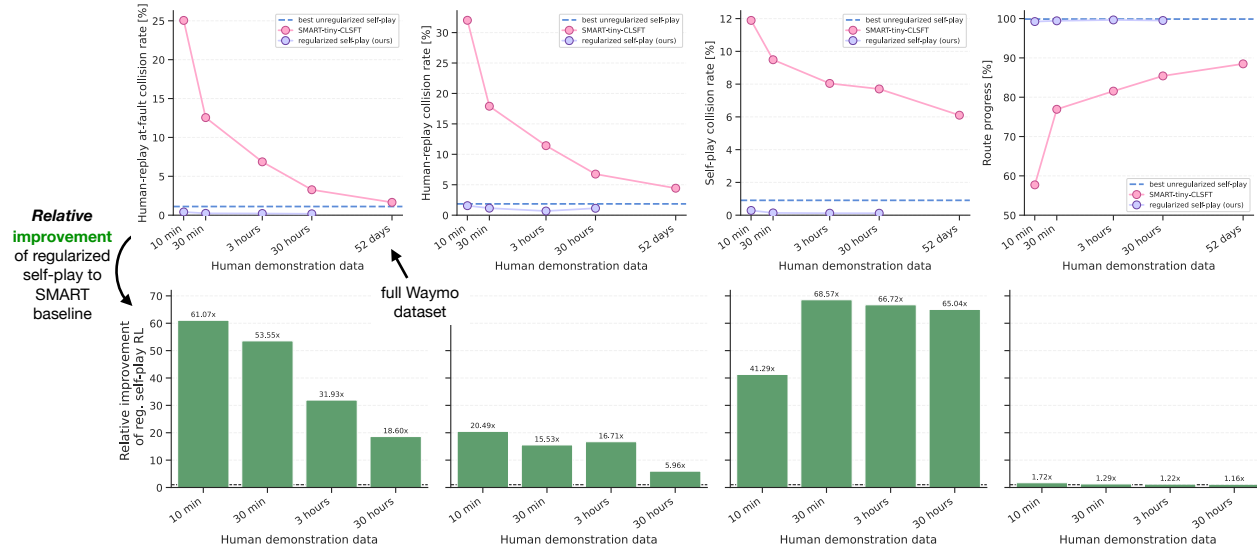


Figure 4: **Scaling laws for human demonstration data.** Performance of *regularized* self-play RL (—●—) and SMART with CAT-K closed-loop fine-tuning (—●—) as a function of the amount of human demonstration data used for training, evaluated in self-play and against human replays. All policies are evaluated on the same 10k held-out scenarios from the Waymo Open validation set [14]. Regularized self-play saturates at roughly three hours of human data, whereas SMART continues to improve across the full WOMB training set. Self-play RL (---) is shown as a horizontal line, since it uses no human demonstrations. The horizontal axis is semi-logarithmic.

Regularized self-play RL outperforms SMART across all data bins. Regularized self-play outperforms SMART-tiny-CLSFT across every data bin and every performance metric. Plain self-play RL is already a strong baseline, achieving a human-replay at-fault collision rate of 1.1% (---). Adding just 10 or 30 minutes of human data (—●—) pulls the policy toward more human-compatible behavior, dropping the collision rate to 0.2%. SMART, trained on 52 days of human demonstration data, reaches an at-fault collision rate of 1.6%, still $8\times$ higher than regularized self-play RL trained on 30 minutes of human data. Part of this gap is unsurprising: the self-play RL agents train on far more total transitions than SMART, 5 billion versus 225 million (Figure 1). But the fundamental difference is that self-play RL policies learn primarily from synthetic experience.

Scaling laws are ... TODO: Something about the shape of the scaling laws of human data? Unsure what I say about this atm, seems pretty saturated at 30 min already.

Table 1: Performance vs. amount of human demonstration data at fixed 50k self-play training maps. Top-3 values per column are highlighted (best , 2nd , 3rd); best value additionally in bold. The unregularized self-play row uses no human demonstrations.

Human demos used	Method	Self-play (test)			Human-replay (test)			
		Coll. (%) ↓	Off-road (%) ↓	Route prog. (%) ↑	Coll. (%) ↓	At-fault (%) ↓	Off-road (%) ↓	Route prog. (%) ↑
10 min	SMART	11.9	55.8	84.5	32.0	25.0	18.6	57.7
30 min	SMART	9.5	55.4	85.8	17.9	12.5	16.8	76.9
3 hours	SMART	8.0	53.6	86.2	11.4	6.9	4.5	81.5
30 hours	SMART	7.7	53.3	86.5	6.8	3.3	1.6	85.4
52.1 days	SMART	6.1	53.5	91.7	4.4	1.6	1.1	88.5
—	unreg. self-play	0.9 ± 0.3	0.5 ± 0.1	99.4 ± 0.1	1.9 ± 0.2	1.1 ± 0.1	1.2 ± 0.2	99.9 ± 0.0
10 min	reg. self-play (ours)	0.3 ± 0.1	0.1 ± 0.1	97.8 ± 0.3	1.6 ± 0.2	0.4 ± 0.1	0.9 ± 0.1	99.2 ± 0.1
30 min	reg. self-play (ours)	0.1 ± 0.1	0.0 ± 0.0	98.0 ± 0.3	1.2 ± 0.1	0.2 ± 0.1	0.2 ± 0.1	99.4 ± 0.1
3 hours	reg. self-play (ours)	0.1 ± 0.1	0.2 ± 0.1	98.5 ± 0.3	0.7 ± 0.1	0.2 ± 0.1	0.2 ± 0.1	99.7 ± 0.1
30 hours	reg. self-play (ours)	0.1 ± 0.0	0.2 ± 0.1	98.3 ± 0.3	1.1 ± 0.1	0.2 ± 0.1	0.5 ± 0.1	99.5 ± 0.1

4.2 The role of metadata in generalization

Scenario *metadata* (road graphs, initial agent positions, and velocity) is typically cheaper than human demonstration data, and it comes with most available driving datasets [7, 14]. Recent work has shown that regularized self-play RL in simulation grounded by target-city metadata can adapt driving policies to new cities [46]. A natural follow-up question is *how much* the diversity provided by metadata matters for training generalizable policies, which is what we explore here. The results in the previous section assume the metadata from 50k training maps, since the focus there was on scaling laws for human demonstration data.

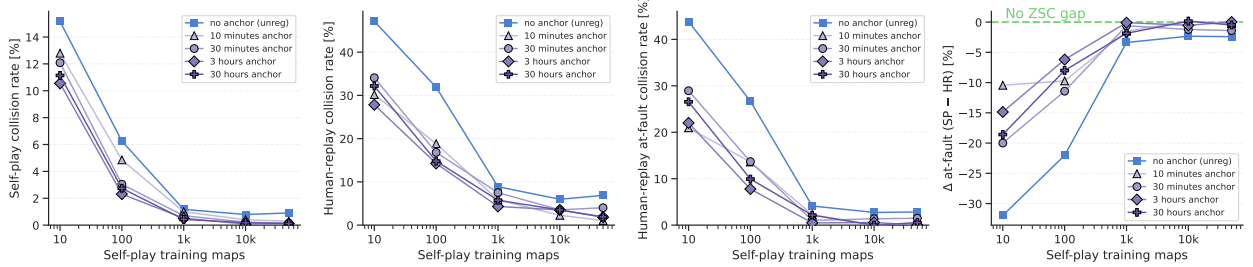


Figure 5: **Scaling laws for scenario metadata.** The unregularized self-play baseline is shown in blue; shades of purple correspond to regularized policies trained with different BC anchors, with darker shades indicating more anchor data. **Left:** collision rate in self-play, where all agents are controlled by the same policy on a held-out validation set. **Center left:** collision rate in human-replay, where only the SDC is controlled and the remaining agents follow their logged trajectories. **Center right:** at-fault collision rate, the fraction of collisions caused by the controlled agent. This metric is more informative than raw collision rate, because replay agents frequently strike the SDC from behind, and such collisions would be unlikely in practice (See cartoon in Figure 2). **Right:** ZSC gap Δ_{ZSC} , the difference in score between the self-play and human-replay settings.

We train regularized and unregularized agents on nested subsets $\mathcal{M}_k$ with $|\mathcal{M}_k| \in \{10, 100, 1,000, 10,000, 50,000\}$ scenarios, holding the BC anchors τ^n and reward function r fixed. This isolates the effect of *scenario diversity*. Each policy is trained for 5 billion steps; Figure 9 confirms convergence on the training set.

To sharpen the comparison between policies, we evaluate on an interactive partition of the held-out set: scenarios filtered to contain dense multi-agent interactions such as merges, unprotected turns, and yielding, where coordination with other drivers is essential (details in Appendix D.1). Most scenarios in the Waymo Open validation set involve

little interaction, and performance on them saturates quickly for all methods, which compresses the differences between policies. The interactive partition preserves those differences and is the more informative signal for zero-shot coordination. Figure 5 and Table 2 report results on this partition; Table 8 reports the same evaluation on the full validation set for completeness.

Scene diversity is key for generalizing to unseen maps and for zero-shot coordination with the human population.

Both plain and regularized self-play improve sharply with the amount of metadata. For plain self-play, the at-fault collision rate drops from 43.8% at 10 scenarios to 2.7% at 10k scenarios, and the human-replay collision rate falls from 47.1% to 6.0% over the same range. Regularized self-play follows the same trend and reaches much lower absolute values: with a 3-hour BC anchor, the at-fault collision rate drops from 22.0% at 10 scenarios to 0.0% at 10k scenarios. The ZSC gap closes in parallel, shrinking from a self-play score that far exceeds the human-replay score to near parity at 10k scenarios (self-play 0.971, human-replay 0.945 for the 3-hour anchor). Diversity during self-play is what drives generalization to unseen maps.

Plain self-play stagnates at 2.7% at-fault collision rate. We see that metadata helps, but only up to a point. The best plain self-play policy, trained on 10k maps, reaches a human-replay collision rate of 6.0% and an at-fault collision rate of 2.7%. Scaling to 50k scenarios yields no further improvement: collision rate ticks up to 6.9% and at-fault rate to 2.8%, both within noise. The bottleneck is no longer scene diversity but the absence of a prior on human-compatible behavior. Plain self-play converges to equilibria that are internally consistent but diverge from how humans actually drive, and more map diversity cannot resolve this discrepancy.

Regularized self-play closes the ZSC gap. Adding a BC anchor removes this ceiling. At 10k scenarios, regularized self-play reaches an at-fault collision rate of 0.0% with 10 minutes of human data, 1.4% with 30 minutes, and 0.6% with 3 hours, all well below the 2.7% plain self-play floor. Human-replay scores climb correspondingly, from 0.900 for the best plain self-play policy to 0.964 with a 10-minute anchor and 50% more maps. The ZSC gap essentially vanishes once metadata reaches 10k scenarios: the self-play and human-replay scores sit within 0.03 of each other across all anchor settings, indicating that the policy transfers to human replays nearly as cleanly as it performs in self-play. The two ingredients are complementary. Metadata supplies the scene diversity needed to generalize, and the BC anchor supplies the behavioral prior needed to coordinate with humans.

Table 2: Scaling evaluation on held-out Waymo scenarios. Self-play metrics are reported on 10k randomly sampled validation scenarios; human-replay metrics on 200 interactive validation scenarios. Top-3 values per column are highlighted (best, 2nd, 3rd); best value additionally in bold. Blue marks the best unregularized value per column.

Self-play maps (metadata)	Anchor data (human demonstrations)	Self-play (test)			Human-replay (test)					
		Score ↑	Coll. (%) ↓	Off-road (%) ↓	Score ↑	Coll. (%) ↓	At-fault (%) ↓	Rear coll. (%) ↓	Route prog. ↑	Lateral L2 ↓
10	0 (unreg.)	0.744 ± 0.009	15.2 ± 0.8	15.6 ± 0.7	0.378 ± 0.007	47.1 ± 0.7	43.8 ± 0.7	33.3 ± 0.7	0.990 ± 0.001	5.85 ± 0.15
100	0 (unreg.)	0.896 ± 0.007	6.3 ± 0.6	4.5 ± 0.4	0.620 ± 0.007	31.9 ± 0.7	26.7 ± 0.6	20.1 ± 0.6	0.989 ± 0.001	5.27 ± 0.15
1k	0 (unreg.)	0.978 ± 0.003	1.2 ± 0.3	0.7 ± 0.1	0.848 ± 0.005	8.9 ± 0.4	4.1 ± 0.3	6.0 ± 0.3	0.982 ± 0.002	3.69 ± 0.14
10k	0 (unreg.)	0.984 ± 0.003	0.8 ± 0.2	0.3 ± 0.1	0.900 ± 0.004	6.0 ± 0.3	2.7 ± 0.2	5.2 ± 0.3	0.994 ± 0.001	2.72 ± 0.09
50k	0 (unreg.)	0.979 ± 0.004	0.9 ± 0.3	0.5 ± 0.1	0.900 ± 0.004	6.9 ± 0.4	2.8 ± 0.2	6.0 ± 0.3	0.998 ± 0.000	2.03 ± 0.06
10	10 minutes	0.738 ± 0.010	12.8 ± 0.8	12.5 ± 0.7	0.513 ± 0.007	30.2 ± 0.6	21.0 ± 0.6	21.9 ± 0.6	0.960 ± 0.003	1.58 ± 0.02
10	30 minutes	0.753 ± 0.010	12.1 ± 0.8	14.1 ± 0.8	0.477 ± 0.007	34.1 ± 0.7	28.9 ± 0.6	21.1 ± 0.6	0.965 ± 0.002	2.22 ± 0.04
10	3 hours	0.765 ± 0.009	10.6 ± 0.7	12.3 ± 0.7	0.464 ± 0.007	27.8 ± 0.6	22.0 ± 0.6	16.7 ± 0.5	0.948 ± 0.002	3.42 ± 0.09
10	30 hours	0.774 ± 0.010	11.1 ± 0.7	11.8 ± 0.7	0.501 ± 0.007	32.2 ± 0.7	26.6 ± 0.6	19.8 ± 0.6	0.979 ± 0.002	1.66 ± 0.02
100	10 minutes	0.910 ± 0.007	4.9 ± 0.5	3.9 ± 0.4	0.753 ± 0.006	18.8 ± 0.5	13.6 ± 0.5	9.3 ± 0.4	0.989 ± 0.001	1.46 ± 0.02
100	30 minutes	0.916 ± 0.006	3.0 ± 0.4	3.5 ± 0.4	0.772 ± 0.006	16.8 ± 0.5	13.7 ± 0.5	4.9 ± 0.3	0.984 ± 0.001	1.20 ± 0.01
100	3 hours	0.918 ± 0.007	2.3 ± 0.4	3.0 ± 0.4	0.797 ± 0.006	14.3 ± 0.5	7.8 ± 0.4	11.1 ± 0.4	0.989 ± 0.001	1.47 ± 0.02
100	30 hours	0.930 ± 0.006	2.7 ± 0.4	2.1 ± 0.3	0.808 ± 0.006	14.8 ± 0.5	9.9 ± 0.4	9.3 ± 0.4	0.984 ± 0.002	1.51 ± 0.02
1k	10 minutes	0.955 ± 0.005	1.0 ± 0.2	0.5 ± 0.2	0.920 ± 0.004	5.9 ± 0.3	2.2 ± 0.2	3.9 ± 0.3	0.992 ± 0.001	1.32 ± 0.02
1k	30 minutes	0.961 ± 0.005	0.7 ± 0.2	0.5 ± 0.1	0.920 ± 0.004	7.5 ± 0.4	1.0 ± 0.1	7.1 ± 0.4	1.000	1.20 ± 0.01
1k	3 hours	0.972 ± 0.004	0.5 ± 0.2	0.4 ± 0.1	0.930 ± 0.004	4.3 ± 0.3	0.5 ± 0.1	3.9 ± 0.3	0.992 ± 0.001	1.31 ± 0.02
1k	30 hours	0.967 ± 0.004	0.4 ± 0.1	0.6 ± 0.1	0.900 ± 0.004	5.8 ± 0.3	2.1 ± 0.2	4.1 ± 0.3	0.985 ± 0.002	1.28 ± 0.02
10k	10 minutes	0.948 ± 0.005	0.4 ± 0.2	0.3 ± 0.1	0.964 ± 0.003	2.3 ± 0.2	0.0	2.3 ± 0.2	0.997 ± 0.000	1.30 ± 0.02
10k	30 minutes	0.951 ± 0.005	0.2 ± 0.1	0.1 ± 0.0	0.952 ± 0.003	3.5 ± 0.3	1.4 ± 0.2	3.5 ± 0.3	0.992 ± 0.001	1.19 ± 0.02
10k	3 hours	0.971 ± 0.004	0.1 ± 0.0	0.0 ± 0.0	0.945 ± 0.003	3.5 ± 0.3	0.6 ± 0.1	2.5 ± 0.2	0.985 ± 0.002	1.45 ± 0.02
10k	30 hours	0.967 ± 0.004	0.2 ± 0.1	0.1 ± 0.1	0.925 ± 0.004	3.5 ± 0.3	0.0	3.5 ± 0.3	0.985 ± 0.002	1.17 ± 0.01
50k	10 minutes	0.928 ± 0.006	0.3 ± 0.1	0.1 ± 0.1	0.954 ± 0.003	1.1 ± 0.1	0.5 ± 0.1	1.1 ± 0.1	0.996 ± 0.000	1.41 ± 0.03
50k	30 minutes	0.962 ± 0.005	0.1 ± 0.1	0.0 ± 0.0	0.936 ± 0.003	4.0 ± 0.3	1.5 ± 0.2	3.6 ± 0.3	0.991 ± 0.001	1.49 ± 0.02
50k	3 hours	0.964 ± 0.004	0.1 ± 0.1	0.2 ± 0.1	0.927 ± 0.004	1.8 ± 0.2	0.0	1.8 ± 0.2	0.985 ± 0.001	1.32 ± 0.02
50k	30 hours	0.967 ± 0.004	0.1 ± 0.0	0.2 ± 0.1	0.951 ± 0.003	1.9 ± 0.2	0.5 ± 0.1	1.5 ± 0.2	0.989 ± 0.001	1.24 ± 0.01

4.3 Behavior analysis

@Daphne: TODO

Characterizing human-compatible driving requires more than a single metric. We report a suite of complementary measures with videos included in the supplementary material.

4.4 Correcting for bad anchor predictions in OOD states

@Daphne: TODO

BC policies are typically reliable in distribution and unreliable outside of it. Since our regularization pulls the RL policy toward the anchor, one concern is that the anchor’s unreliability in out-of-distribution (OOD) states could contaminate training: the RL policy receives a noisy, sometimes misleading gradient signal. We see evidence of this in our training dynamics. Regularized runs with smaller anchors (10 or 30 minutes of human data) show visibly noisier learning curves and take longer to converge than runs anchored on larger datasets, consistent with the anchor producing unreliable predictions more often during rollouts. Nonetheless, asymptotic performance is comparable across anchor sizes (Section 4.1, Table ??): self-play RL appears to correct for anchor noise over time, at the cost of additional training steps.

5 Conclusion & limitations

The mixing of human data with synthetic experience generated through self-play is, for now, an art. This paper is a small step toward turning it into a science. We show that a pinch of human data, roughly 30 minutes of demonstrations, suffices to train human-compatible driving agents, when used as a behavioral cloning anchor to regularize self-play RL. This is three orders of magnitude less than state-of-the-art imitation learning baselines, eliminates the need for reward shaping, and is achieved at a fraction of the training time. The broader implication is that, wherever simulation is cheap, human data may be best used **not** as the primary training signal but as a *lightweight anchor* that steers policies away from effective-but-alien equilibria.

This empirical result raises a deeper question that we have only touched the surface of: what is the *complementary value* of a bit of human data given a set of synthetic experience? Can we quantify this? We can loosely intuit that self-play trains more robust agents because it covers a wider distribution of states, and indeed, our self-play agents see 5 billion transitions against 225 million for the IL baseline (Figure 1). But sheer count is a crude explanation; not all transitions are equally informative. Recent work on Epiplexity [15] takes a step toward formalizing this, conceptually defining the value of data as the amount of learnable structure it contains. Translating such notions into practical tools for selecting or filtering data remains open.

As for limitations, while regularized self-play improves on prior work in multiple dimensions, several aspects remain to be improved. First, our evaluation relies on human-replay as a proxy for deployment, and it is currently unclear to what extent performance in the self-play and human-replay settings *exactly translates* to behavior on the road; closing the sim-to-real gap remains future work. Second, our best policy reaches a near-zero at-fault collision rate, but not a zero overall collision rate: replay agents can strike the SDC from behind in situations the policy cannot reasonably avoid. What counts as a fair collision-avoidance benchmark beyond at-fault heuristics like those used in this paper is itself a difficult open question, both in industry and academia [36].

Acknowledgments

We thank the authors of CAT-K [50] for generously sharing the weights of their best SMART-tiny-CLSFT checkpoint, which we use as the imitation learning baseline throughout the paper, and their code, which we use as a baseline for the scaling law experiments. We thank Wael Doulazmi for providing the basis for the SMART evaluation code and sanity checking the baseline results. We also thank Luke Rowe, Rodrigue de Schaetzen, and Roger Girgis for feedback on some early results and various interesting discussions on the topic of end-to-end driving and self-play. We thank Momchil Tomov for a helpful discussion on evals and metrics for evaluating human-likeness and compatibility in driving.

Daphne Cornelisse is partially supported by the Cooperative AI Foundation and a Chishiki-AI SCIFE Fellowship. [FUNDING]

References

- [1] Anton Bakhtin, David Wu, Adam Lerer, and Noam Brown. No-press diplomacy from scratch. *Advances in Neural Information Processing Systems*, 34:18063–18074, 2021.
- [2] Anton Bakhtin, David J Wu, Adam Lerer, Jonathan Gray, Athul Paul Jacob, Gabriele Farina, Alexander H Miller, and Noam Brown. Mastering the game of no-press Diplomacy via human-regularized reinforcement learning and planning. In *International Conference on Learning Representations*, 2023. arXiv:2210.05492.
- [3] Mustafa Baniodeh, Kratharth Goel, Scott Ettinger, Carlos Fuertes, Ari Seff, Tim Shen, Cole Gulino, Chenjie Yang, Ghassen Jerfel, Dokook Choe, Rui Wang, Vinutha Kallem, Sergio Casas, Rami Al-Rfou, Benjamin Sapp, and Dragomir Anguelov. Scaling laws of motion forecasting and planning: A technical report. *arXiv preprint arXiv:2506.08228*, 2025.
- [4] Nolan Bard, Jakob N Foerster, Sarath Chandar, Neil Burch, Marc Lanctot, H Francis Song, Emilio Parisotto, Vincent Dumoulin, Subhodeep Moitra, Edward Hughes, et al. The hanabi challenge: A new frontier for ai research. *Artificial Intelligence*, 280:103216, 2020.
- [5] Mariusz Bojarski, Davide Del Testa, Daniel Dworakowski, Bernhard Firner, Beat Flepp, Praseen Goyal, Lawrence D Jackel, Mathew Monfort, Urs Muller, Jiakai Zhang, et al. End to end learning for self-driving cars. *arXiv preprint arXiv:1604.07316*, 2016.
- [6] H Caesar, V Bankiti, AH Lang, S Vora, VE Liong, Q Xu, A Krishnan, Y Pan, G Baldan, and O Beijbom. nuscenes: A multimodal dataset for autonomous driving. arxiv. 2019.
- [7] Holger Caesar, Juraj Kabzan, Kok Seang Tan, Whye Kit Fong, Eric Wolff, Alex Lang, Luke Fletcher, Oscar Beijbom, and Sammy Omari. nuplan: A closed-loop ml-based planning benchmark for autonomous vehicles. *arXiv preprint arXiv:2106.11810*, 2021.
- [8] Wei-Jer Chang, Akshay Rangesh, Kevin Joseph, Matthew Strong, Masayoshi Tomizuka, Yihan Hu, and Wei Zhan. SPACeR: Self-play anchoring with centralized reference models. *arXiv preprint arXiv:2510.18060*, 2025.
- [9] Daphne Cornelisse and Eugene Vinitsky. Human-compatible driving partners through data-regularized self-play reinforcement learning. In *Reinforcement Learning Journal*, 2024. arXiv:2403.19648.
- [10] Daphne Cornelisse, Aarav Pandya, Kevin Joseph, Joseph Suárez, and Eugene Vinitsky. Building reliable sim driving agents by scaling self-play. *arXiv preprint arXiv:2502.14706*, 2025.
- [11] Daphne Cornelisse, Spencer Cheng, Pragnay Mandavilli, Julian Hunt, Kevin Joseph, Wael Doulazmi, Valentin Charraut, Aditya Gupta, Joseph Suarez, and Eugene Vinitsky. PufferDrive: A fast and friendly driving simulator for training and evaluating RL agents, 2025. URL <https://github.com/Emerge-Lab/PufferDrive>.
- [12] Marco Cusumano-Towner, David Hafner, Alex Hertzberg, Brody Huval, Aleksei Petrenko, Eugene Vinitsky, Erik Wijmans, Taylor Killian, Stuart Bowers, Ozan Sener, Philipp Krähenbühl, and Vladlen Koltun. Robust autonomy emerges from self-play. *arXiv preprint arXiv:2502.03349*, 2025.
- [13] Daniel Dauner, Marcel Hallgarten, Tianyu Li, Xinshuo Weng, Zhiyu Huang, Zetong Yang, Hongyang Li, Igor Gillitschenski, Boris Ivanovic, Marco Pavone, et al. Navsim: Data-driven non-reactive autonomous vehicle simulation and benchmarking. *Advances in Neural Information Processing Systems*, 37:28706–28719, 2024.
- [14] Scott Ettinger, Shuyang Cheng, Benjamin Caine, Chenxi Liu, Hang Zhao, Sabeek Pradhan, Yuning Chai, Ben Sapp, Charles R Qi, Yin Zhou, et al. Large scale interactive motion forecasting for autonomous driving: The waymo open motion dataset. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9710–9719, 2021.

- [15] Marc Finzi, Shikai Qiu, Yiding Jiang, Pavel Izmailov, J Zico Kolter, and Andrew Gordon Wilson. From entropy to epiplexity: Rethinking information for computationally bounded intelligence. *arXiv preprint arXiv:2601.03220*, 2026.
- [16] Junru Gu, Chen Sun, and Hang Zhao. Densetnt: End-to-end trajectory prediction from dense goal sets. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 15303–15312, 2021.
- [17] Cole Gulino, Justin Fu, Wenjie Luo, George Tucker, Eli Bronstein, Yiren Lu, Jean Harb, Xinlei Pan, Yan Wang, Xiangyu Chen, et al. Waymax: An accelerated, data-driven simulator for large-scale autonomous driving research. *Advances in Neural Information Processing Systems*, 36:7730–7742, 2023.
- [18] Hengyuan Hu, Adam Lerer, Alex Peysakhovich, and Jakob Foerster. “other-play” for zero-shot coordination. In *International conference on machine learning*, pages 4399–4410. PMLR, 2020.
- [19] Hengyuan Hu, David J Wu, Adam Lerer, Jakob Foerster, and Noam Brown. Human-ai coordination via human-regularized search and learning. *arXiv preprint arXiv:2210.05125*, 2022.
- [20] Zhiyu Huang, Zixu Zhang, Ameya Vaidya, Yuxiao Chen, Chen Lv, and Jaime Fernández Fisac. Versatile behavior diffusion for generalized traffic agent simulation. *arXiv preprint arXiv:2404.02524*, 2024.
- [21] Athul Paul Jacob, David J Wu, Gabriele Farina, Adam Lerer, Hengyuan Hu, Anton Bakhtin, Jacob Andreas, and Noam Brown. Modeling strong and human-like gameplay with KL-regularized search. In *International Conference on Machine Learning*, pages 9695–9728. PMLR, 2022.
- [22] Chiyu Jiang, Andre Cornman, Cheolho Park, Benjamin Sapp, Yin Zhou, Dragomir Anguelov, et al. Motiondiffuser: Controllable multi-agent motion prediction using diffusion. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 9644–9653, 2023.
- [23] Saman Kazemkhani, Aarav Pandya, Daphne Cornelisse, Brennan Shacklett, and Eugene Vinitsky. Gpudrive: Data-driven, multi-agent driving simulation at 1 million fps. *arXiv preprint arXiv:2408.01584*, 2024.
- [24] W Bradley Knox, Alessandro Allievi, Holger Banzhaf, Felix Schmitt, and Peter Stone. Reward (mis) design for autonomous driving. *Artificial Intelligence*, 316:103829, 2023.
- [25] Joel Z Leibo, Edward Hughes, Marc Lanctot, and Thore Graepel. Autocurricula and the emergence of innovation from social interaction: A manifesto for multi-agent intelligence research. *arXiv preprint arXiv:1903.00742*, 2019.
- [26] Yiren Lu, Justin Fu, George Tucker, Xinlei Pan, Eli Bronstein, Rebecca Roelofs, Benjamin Sapp, Brandyn White, Aleksandra Faust, Shimon Whiteson, et al. Imitation is not enough: Robustifying imitation with reinforcement learning for challenging driving scenarios. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 7553–7560. IEEE, 2023.
- [27] Nico Montali, John Lambert, Paul Mougin, Alex Kuefler, Nicholas Rhinehart, Michelle Li, Cole Gulino, Tristan Emrich, Zoey Yang, Shimon Whiteson, et al. The waymo open sim agents challenge. *Advances in Neural Information Processing Systems*, 36:59151–59171, 2023.
- [28] Nigamaa Nayakanti, Rami Al-Rfou, Aurick Zhou, Kratarth Goel, Khaled S Refaat, and Benjamin Sapp. Wayformer: Motion forecasting via simple & efficient attention networks. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2980–2987. IEEE, 2023.
- [29] Jiquan Ngiam, Benjamin Caine, Vijay Vasudevan, Zhengdong Zhang, Hao-Tien Lewis Chiang, Jeffrey Ling, Rebecca Roelofs, Alex Bewley, Chenxi Liu, Ashish Venugopal, et al. Scene transformer: A unified architecture for predicting multiple agent trajectories. *arXiv preprint arXiv:2106.08417*, 2021.
- [30] Zhenghao Peng, Wenjie Luo, Yiren Lu, Tianyi Shen, Cole Gulino, Ari Seff, and Justin Fu. Improving agent behaviors with RL fine-tuning for autonomous driving. In *Computer Vision - ECCV 2024 - 18th European Conference*, volume 15083 of *Lecture Notes in Computer Science*, pages 165–181. Springer, 2024.

- [31] Julien Perolat, Bart De Vylder, Daniel Hennes, Eugene Tarassov, Florian Strub, Vincent de Boer, Paul Muller, Jerome T Connor, Neil Burch, Thomas Anthony, et al. Mastering the game of Stratego with model-free multiagent reinforcement learning. *Science*, 378(6623):990–996, 2022.
- [32] Jonah Philion, Xue Bin Peng, and Sanja Fidler. Trajeglisch: Traffic modeling as next-token prediction. *arXiv preprint arXiv:2312.04535*, 2023.
- [33] Dean A Pomerleau. Alvin: An autonomous land vehicle in a neural network. *Advances in neural information processing systems*, 1, 1988.
- [34] Jinkai Qiu, Alessandro Saviolo, Chaojie Wang, Mingke Wang, and Xiaoyu Huang. Heterogeneous self-play for realistic highway traffic simulation. 2026. URL <https://arxiv.org/abs/2604.16406>.
- [35] Tim Salzmann, Boris Ivanovic, Punarjay Chakravarty, and Marco Pavone. Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XVIII 16*, pages 683–700. Springer, 2020.
- [36] John M Scanlon, Kristofer D Kusano, Johan Engstrom, and Trent Victor. Collision avoidance effectiveness of an automated driving system using a human driver behavior reference model in reconstructed fatal collisions. In *WCX SAE World Congress Experience*. SAE Technical Paper, 2026.
- [37] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [38] Ari Seff, Brian Cera, Dian Chen, Mason Ng, Aurick Zhou, Nigamaa Nayakanti, Khaled S Refaat, Rami Al-Rfou, and Benjamin Sapp. Motionlm: Multi-agent motion forecasting as language modeling. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 8579–8590, 2023.
- [39] Shaoshuai Shi, Li Jiang, Dengxin Dai, and Bernt Schiele. Motion transformer with global intention localization and local movement refinement. *Advances in Neural Information Processing Systems*, 35:6531–6543, 2022.
- [40] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- [41] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.
- [42] Samuel Sokota, Eugene Vinitsky, Hengyuan Hu, J Zico Kolter, and Gabriele Farina. Superhuman ai for stratego using self-play reinforcement learning and test-time search. *arXiv preprint arXiv:2511.07312*, 2025.
- [43] Joseph Suarez. PufferLib: Making reinforcement learning libraries and environments play nice. *arXiv preprint arXiv:2406.12905*, 2024.
- [44] Balakrishnan Varadarajan, Ahmed Hefny, Avikalp Srivastava, Khaled S Refaat, Nigamaa Nayakanti, Andre Cornman, Kan Chen, Bertrand Douillard, Chi Pang Lam, Dragomir Anguelov, et al. Multipath++: Efficient information fusion and trajectory aggregation for behavior prediction. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 7814–7821. IEEE, 2022.
- [45] Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *nature*, 575(7782):350–354, 2019.
- [46] Zilin Wang, Saeed Rahmani, Daphne Cornelisse, Bidipta Sarkar, Alexander David Goldie, Jakob Nicolaus Foerster, and Shimon Whiteson. Learning to drive in new cities without human demonstrations. *arXiv preprint arXiv:2602.15891*, 2026.

- [47] Benjamin Wilson, William Qi, Tanmay Agarwal, John Lambert, Jagjeet Singh, Siddhesh Khandelwal, Bowen Pan, Ratnesh Kumar, Andrew Hartnett, Jhony Kaesemodel Pontes, et al. Argoverse 2: Next generation datasets for self-driving perception and forecasting. *arXiv preprint arXiv:2301.00493*, 2023.
- [48] Wei Wu, Xiaoxin Feng, Ziyang Gao, and Yuheng Kan. Smart: Scalable multi-agent real-time motion generation via next-token prediction. *Advances in Neural Information Processing Systems*, 37:114048–114071, 2024.
- [49] Tenny Yin, Zhiting Mei, Zhonghe Zheng, Miyu Yamane, David Wang, Jade Sceats, Samuel M Bateman, Lihan Zha, Apurva Badithela, Ola Shorinwa, and Anirudha Majumdar. PlayWorld: Learning robot world models from autonomous play. *arXiv preprint arXiv:2603.09030*, 2026.
- [50] Zhejun Zhang, Peter Karkus, Maximilian Igl, Wenhao Ding, Yuxiao Chen, Boris Ivanovic, and Marco Pavone. Closed-loop supervised fine-tuning of tokenized traffic models. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [51] Hang Zhao, Jiyang Gao, Tian Lan, Chen Sun, Ben Sapp, Balakrishnan Varadarajan, Yue Shen, Yi Shen, Yuning Chai, Cordelia Schmid, et al. Tnt: Target-driven trajectory prediction. In *Conference on Robot Learning*, pages 895–904. PMLR, 2021.
- [52] Ziyuan Zhong, Davis Rempe, Yuxiao Chen, Boris Ivanovic, Yulong Cao, Danfei Xu, Marco Pavone, and Baishakhi Ray. Language-guided traffic simulation via scene-level diffusion. In *Conference on Robot Learning*, pages 144–177. PMLR, 2023.
- [53] Ziyuan Zhong, Davis Rempe, Danfei Xu, Yuxiao Chen, Sushant Veer, Tong Che, Baishakhi Ray, and Marco Pavone. Guided conditional diffusion for controllable traffic simulation. In *2023 IEEE international conference on robotics and automation (ICRA)*, pages 3560–3566. IEEE, 2023.
- [54] Zikang Zhou, Luyao Ye, Jianping Wang, Kui Wu, and Kejie Lu. Hivt: Hierarchical vector transformer for multi-agent motion prediction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8823–8833, 2022.
- [55] Zikang Zhou, Jianping Wang, Yung-Hui Li, and Yu-Kai Huang. Query-centric trajectory prediction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17863–17873, 2023.

A Simulation environment and design

A.1 World initialization from scenario metadata

We use PufferDrive 2.0 for simulation and training [11]. PufferDrive is a batched simulator that runs many environments in parallel, reaching 300k steps per second (SPS) on a single consumer-grade GPU. We initialize environments using the Waymo Open Motion Dataset (WOMD) [14], which provides a large set of multi-agent traffic scenarios. Each scenario supplies the metadata we need: the roadgraph, a variable number of agents (cars, cyclists, and pedestrians), and other objects in the scene. This information is the output of a perception stack, so we operate directly on these clean features (in bounding-box world).

Each scenario is 9 seconds long and discretized into 90 steps. We take each logged agent’s initial position ($t = 0$) as its starting position in the scene, and its last valid logged position ($t = T$) as its goal, which lets us goal-condition the agents. The full Waymo training dataset contains 500k scenarios, but in this paper we use at most 50k of the randomly sampled scenarios. When constructing the environments, we randomly sample scenarios from WOMD until we hit a target number of agents (e.g., on an NVIDIA RTX 4080 with 16GB of memory, we keep adding environments until we reach 1024 agents).

A.2 Observation space

We take a decentralized approach and provide every agent with a partial view of the environment. This is similar to the observation space of prior related works, such as GIGAFLOW [12], and GPU Drive [23]. At each timestep, an agent receives the concatenation of three feature blocks: an ego block describing its own state, a partner block describing the $N_p = 31$ closest other agents within a 50 m radius, and a road block describing up to $N_r = 128$ nearby road segments drawn from a 21×21 grid of $5 \text{ m} \times 5 \text{ m}$ cells centered on the agent. Missing slots (fewer partners or road segments than the maximum) are zero-padded. Tables 3, 4, and 5 list the features in each block. All positions and headings are expressed in the agent’s local frame, so the observation is invariant to the global pose of the scene.

Table 3: Ego features (12 values). Features 0–3 expose the sampled conditioning variables to the policy so it can modulate its behavior as a function of λ and the reward weights (Section 4.2).

Idx	Feature	Normalization	Description
0	λ	—	Human-regularization coefficient
1	r_{coll}	—	Sampled collision reward
2	r_{off}	—	Sampled off-road reward
3	r_{goal}	—	Sampled goal reward
4	Δx_{goal}	$\times 0.005$	Goal position (ego frame), longitudinal
5	Δy_{goal}	$\times 0.005$	Goal position (ego frame), lateral
6	signed speed	/ 100 m/s	Speed projected onto heading
7	vehicle width	/ 15 m	Ego bounding-box width
8	vehicle length	/ 30 m	Ego bounding-box length
9	collision flag	{0, 1}	1 if currently colliding
10	(reserved)	—	Zero placeholder
11	entity type	/ 3	Vehicle (1), pedestrian (2), cyclist (3)

Table 4: Partner features (7 values $\times$ 31 partners = 217 values). Partners are ordered by index and filtered to those within 50 m of the ego agent. All positions and headings are in the ego frame.

Idx	Feature	Normalization	Description
0	Δx	$\times 0.02$	Partner position, longitudinal
1	Δy	$\times 0.02$	Partner position, lateral
2	partner width	/ 15 m	Partner bounding-box width
3	partner length	/ 30 m	Partner bounding-box length
4	$\cos(\Delta\psi)$	—	Relative heading, cosine component
5	$\sin(\Delta\psi)$	—	Relative heading, sine component
6	partner signed speed	/ 100 m/s	Signed speed along partner’s heading

Table 5: Road-segment features (7 values $\times$ 128 segments = 896 values). Segments are drawn from a 21×21 grid of 5 m cells centered on the ego agent, and include road lanes, road lines, and road edges. Each segment is described by the midpoint, length, and orientation of a single polyline segment.

Idx	Feature	Normalization	Description
0	midpoint x	$\times 0.02$	Segment midpoint, longitudinal (ego frame)
1	midpoint y	$\times 0.02$	Segment midpoint, lateral (ego frame)
2	segment length	/ 100 m	Length of the polyline segment
3	segment width	/ 100 m	Fixed nominal width (0.1 m)
4	$\cos(\theta)$	—	Segment orientation in ego frame
5	$\sin(\theta)$	—	Segment orientation in ego frame
6	segment type	{0, 1, 2}	Road lane (0), road line (1), road edge (2)

The total observation vector has dimension $12 + 7 \times 31 + 7 \times 128 = 1,125$. When sensor noise ablations are enabled, zero-mean Gaussian noise is added independently to the partner position, size, and speed features.

A.3 Actions and dynamics

We use two different dynamics models in this paper. Both use a discretized action space.

Delta-local dynamics (regularized experiments). The action is a triple $(\Delta x, \Delta y, \Delta\psi)$ in the agent’s local frame at time t . Translation is rotated into the world frame and added to the position; heading is updated directly:

$$x_{t+1} = x_t + \cos(\psi_t) \Delta x - \sin(\psi_t) \Delta y, \quad (4)$$

$$y_{t+1} = y_t + \sin(\psi_t) \Delta x + \cos(\psi_t) \Delta y, \quad (5)$$

$$\psi_{t+1} = \text{wrap}(\psi_t + \Delta\psi). \quad (6)$$

Velocity is reported as the world-frame displacement divided by $\Delta t = 0.1$ s. Each component is bounded: $\Delta x \in [-2.0, 2.0]$ m, $\Delta y \in [-0.1, 0.1]$ m, and $\Delta\psi \in [-\pi/12, \pi/12]$. The three dimensions are binned independently into $127 \times 127 \times 256$ values, giving step sizes of ≈ 0.032 m, ≈ 0.0016 m, and ≈ 0.0041 rad. Delta-local dynamics are kinematically unconstrained: the agent can translate laterally without rotating, and angular velocity is decoupled from speed. Two consecutive logged poses determine a delta-local action exactly, up to bin width; the same poses do not determine a unique bicycle-model action, since the inversion involves a division by speed and an arctangent and is ill-conditioned at low speeds. Wang et al. [46] report the same effect empirically. Better action recovery means better training data for the anchor, which is why we use delta-local dynamics in the regularized experiments.

Classic dynamics (unregularized experiments). The action is a pair (acceleration, steering), passed through a kinematic bicycle model with wheelbase ℓ equal to the vehicle length. Longitudinal speed is integrated under acceleration; the heading evolves with yaw rate $\dot{\psi} = v \cos(\beta) \tan(\delta)/\ell$, where δ is the steering angle and $\beta =$

$\tanh(\frac{1}{2} \tan \delta)$ is a slip approximation. Acceleration is bounded to $[-4, 4]$ m/s² and steering to $[-1, 1]$ rad, binned into 21 and 31 values respectively; speed is clipped to 100 m/s.

The bicycle model places hard constraints on turning radius and couples lateral motion to heading. Under these constraints, unregularized policies produce kinematically plausible trajectories. Under delta-local dynamics, they do not: without a human anchor, the policy finds behaviors that satisfy the reward while ignoring physical constraints on vehicle motion.

A.4 Collecting human demonstrations

The behavioral cloning (BC) anchor is trained on observation-action pairs (o_t, a_t) . Therefore, we need actions that (i) live in the simulator’s action space and (ii) reproduce the logged motion when applied through the simulator’s dynamics. We construct the dataset for behavioral cloning in two steps.

Step 1: Inferring actions from the logs. For each timestep t , we project the world-frame displacement into the agent’s local frame at t :

$$\Delta x_t = \cos(\psi_t)(x_{t+1} - x_t) + \sin(\psi_t)(y_{t+1} - y_t), \quad (7)$$

$$\Delta y_t = -\sin(\psi_t)(x_{t+1} - x_t) + \cos(\psi_t)(y_{t+1} - y_t), \quad (8)$$

$$\Delta \psi_t = \text{wrap}(\psi_{t+1} - \psi_t). \quad (9)$$

Each triple is clipped to the action bounds and snapped to the nearest discrete bin. Timesteps where either t or $t + 1$ is flagged invalid in the log are marked as missing and excluded from training.

Step 2: Replaying actions through the simulator. If we train a policy directly on logged (observation, inferred action) pairs, it never sees the observations our simulator would actually produce — our road representation, collision geometry, and neighbor perception don’t exactly match the logging platform. To fix this mismatch, we replay the inferred action sequence through the simulator and record the observation it computes at each step. We then train the anchor on these (simulator observation, inferred action) pairs. Over 91 steps, small numerical errors can build up, influencing the data quality. So after each simulator step, we teleport the controlled agent back to its logged pose before computing the next observation. Other agents are not teleported: they move under their own inferred actions, just as they will during self-play. This keeps the scene geometry consistent while stopping the cloned agent from drifting off course.

A.5 Reward function

We use a sparse reward: $r^i = +1$ if agent i reaches its goal within $\delta = 2$ meters before the episode ends, -1 on collision or going off-road, and 0 otherwise. We deliberately omit dense shaping terms so that safe and human-compatible behaviors can emerge from regularization.

B Training

B.1 Behavioral cloning anchor policies

Each anchor τ_n is trained by minimizing the negative log-likelihood of the logged actions under the factorized discrete action distribution described in Appendix A.4. We use only the SDC trajectory from each scenario, as it is consistently the highest-quality demonstration (other agents are products of the perception stack and exhibit more noise); each scenario therefore contributes roughly 9 seconds of human data.

We train with Adam at a learning rate of 10^{-4} and a batch size of 2048 for up to 2000 epochs. Early stopping is applied on the held-out validation loss: training stops if the validation loss fails to improve for 100 consecutive epochs. Table 6 reports open- and closed-loop metrics for each anchor.

Table 6: BC anchor evaluation. Open-loop metrics are computed on the held-out validation set; closed-loop metrics are averaged over validation scenarios.

Human data (h)	Open-loop		Closed-loop self-play		Closed-loop human-replay (SDC only)	
	Acc. (%)	Loss	Route prog.	Score	Route prog.	Score
0.2	69.4	1.923	0.600 ± 0.013	0.178 ± 0.013	0.680 ± 0.007	0.253 ± 0.010
0.5	87.8	0.663	0.743 ± 0.011	0.318 ± 0.015	0.763 ± 0.007	0.366 ± 0.011
3.0	93.8	0.202	0.767 ± 0.012	0.420 ± 0.016	0.783 ± 0.007	0.460 ± 0.011
30.0	94.1	0.184	0.811 ± 0.010	0.439 ± 0.017	0.776 ± 0.008	0.446 ± 0.011

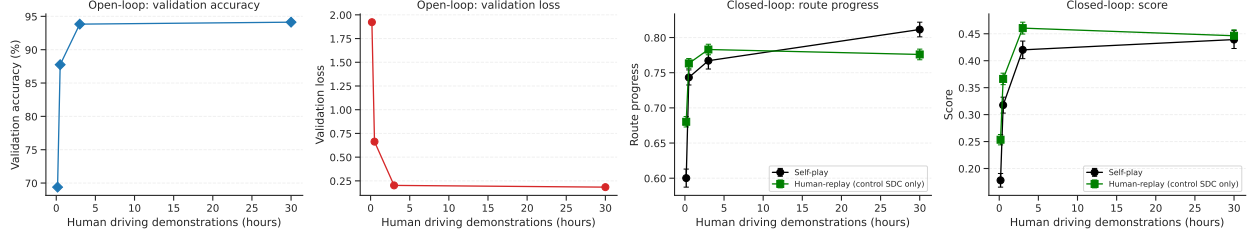


Figure 6: Open- and closed-loop performance of the anchor BC policies as a function of human demonstration data.

B.2 Self-play reinforcement learning

B.3 λ -conditioned regularized self-play RL

Our approach combines two ideas from prior work. Hu et al. [19] show that regularizing an RL policy toward a BC anchor and conditioning the policy on the regularization strength allows a single policy to express a range of human-compatible skill levels. Cusumano-Towner et al. [12] show that conditioning a policy on its sampled reward weights, rather than on a summary statistic, is an effective way to distill a diverse population of behaviors into a single steerable policy. We combine these ideas: we sample a regularization weight λ per episode from a uniform distribution and expose its value to the policy as part of the observation. This induces a heterogeneous population during self-play: agents must coordinate with partners of varying human-likeness, while keeping the trained policy steerable at inference without retraining.

B.3.1 Objective

Let π_θ denote the RL policy with parameters θ , and let τ_n be the fixed BC anchor trained on n scenarios.

Sampling. At the start of each episode, every agent i is assigned a regularization weight

$$\lambda_i \sim \text{Uniform}(0, 1), \quad (10)$$

held fixed for the duration of the episode and resampled on reset.

Regularization loss. We regularize π_θ toward τ_n by penalizing the KL divergence between the two policies on states visited during the rollout. For a minibatch of M transitions $\{(o_j, \lambda_j)\}_{j=1}^M$:

$$\mathcal{L}_{\text{reg}}(\theta) = \frac{1}{M} \sum_{j=1}^M \lambda_j D_{\text{KL}}(\tau_n(\cdot \mid o_j) \parallel \pi_\theta(\cdot \mid o_j, \lambda_j)), \quad (11)$$

where

$$D_{\text{KL}}(\tau_n \parallel \pi_\theta) = \sum_a \tau_n(a \mid o) [\log \tau_n(a \mid o) - \log \pi_\theta(a \mid o, \lambda)]. \quad (12)$$

Transitions with larger λ_j are pulled more strongly toward the anchor; when $\lambda_j = 0$ the regularization vanishes and the policy learns from the PPO objective alone. Because λ_j is part of the observation, π_θ learns to modulate its behavior as a continuous function of λ .

Full training objective. The standard PPO objective combines a clipped surrogate policy-gradient loss $\mathcal{L}_{\text{pg}}$, a value-function loss $\mathcal{L}_v$, and an entropy bonus H . The full training objective is

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{pg}} + c_v \mathcal{L}_v - c_H H + \mathcal{L}_{\text{reg}}, \quad (13)$$

where c_v and c_H are the value and entropy coefficients. No additional scalar multiplies $\mathcal{L}_{\text{reg}}$: the per-sample weighting by λ_j already controls the regularization strength.

Inference. At test time, λ can be set to any value in $[0, 1]$ to continuously control the degree of human-likeness: $\lambda = 1$ produces the most human-like behavior, and $\lambda = 0$ recovers the unregularized self-play policy.

B.3.2 Effect of regularization on the policy gradient

Consider the gradient of the regularization loss with respect to θ .

Per-sample gradient. For a single rollout transition with observation o and conditioning value λ , the regularization loss contributes

$$\nabla_\theta \lambda D_{\text{KL}}(\tau_n(\cdot | o) \parallel \pi_\theta(\cdot | o, \lambda)) = -\lambda \sum_a \tau_n(a | o) \nabla_\theta \log \pi_\theta(a | o, \lambda). \quad (14)$$

This is a weighted behavioral cloning gradient: each action a is reinforced in proportion to the anchor’s probability $\tau_n(a | o)$, scaled by λ . When λ is large, the gradient strongly pushes π_θ toward the anchor; when $\lambda = 0$, the regularization term vanishes and only the PPO objective drives learning.

Interaction with the PPO gradients. The PPO surrogate loss produces a gradient of the form

$$\nabla_\theta \mathcal{L}_{\text{pg}} \propto -\hat{A}(o, a) \nabla_\theta \log \pi_\theta(a | o, \lambda), \quad (15)$$

where $\hat{A}$ is the estimated advantage and a is the action taken during the rollout. The entropy bonus contributes

$$-c_H \nabla_\theta H[\pi_\theta] = c_H \sum_a (1 + \log \pi_\theta(a | o, \lambda)) \nabla_\theta \log \pi_\theta(a | o, \lambda). \quad (16)$$

Combining all three terms gives

$$\underbrace{-\hat{A}(o, a) \nabla_\theta \log \pi_\theta(a | o, \lambda)}_{\text{policy gradient}} \quad (17)$$

$$+ \underbrace{c_H \sum_{a'} (1 + \log \pi_\theta(a' | o, \lambda)) \nabla_\theta \log \pi_\theta(a' | o, \lambda)}_{\text{entropy bonus}} \quad (18)$$

$$- \underbrace{\lambda \sum_{a'} \tau_n(a' | o) \nabla_\theta \log \pi_\theta(a' | o, \lambda)}_{\text{KL regularization}}. \quad (19)$$

Entropy bonus vs. KL regularization. The entropy bonus and the KL term are structurally similar: both sum over the full action space and both act independently of the collected rewards. However, they pull the policy in different directions. The entropy bonus weights each action by $1 + \log \pi_\theta(a)$, pushing probability mass toward the uniform distribution — an *untargeted* form of exploration. The KL term weights each action by $\tau_n(a)$, pushing mass toward the anchor’s preferred actions — a *targeted* form of regularization that steers the policy toward a reference distribution learned from human data.

B.3.3 Baseline

We train the unregularized self-play baselines ($\lambda = 0$ throughout training) for 5 billion steps each on $\{10, 100, 1,000, 10,000, 50,000\}$ maps, which is sufficient for convergence (Figure 9).

B.3.4 Data filtering

We train only on transitions from agents that have not yet reached their goal. Once an agent is within δ of its goal, it is removed from the scene and its remaining transitions are excluded from training.

B.3.5 Hyperparameters

We use PufferLib [43] for training. Hyperparameters are listed in Table 7. We use the native Protein software for sweeps.

Table 7: PPO training hyperparameters.

<i>Architecture</i>		<i>Training</i>		<i>Environment & Rewards</i>	
Input size	64	Total timesteps	10B	Number of agents	1,024
Hidden size	256	Batch size	524,288	Number of workers	16
RNN type	LSTM	Minibatch size	32,768	Episode length	91 steps
RNN input size	256	Rollout horizon	32	Timestep Δt	0.1 s
RNN hidden size	256	Update epochs	1	Goal radius	2.5 m
		Learning rate	4.26×10^{-3}	Action space	Discrete
		LR schedule	Linear annealing	Dynamics model	Delta-local
		Adam β_1	0.9	Goal reward	+1.0
		Adam β_2	0.999	Collision penalty	-1.0
		Adam ϵ	10^{-8}	Off-road penalty	-1.0
		Clip coefficient	0.2		
		Entropy coefficient	0.001		
		VF coefficient	2.0		
		VF clip	0.2		
		GAE λ	0.95		
		Discount γ	0.99		
		Max gradient norm	1.0		
		Priority α	0.85		
		Priority β_0	0.85		
		V-trace c clip	1.0		
		V-trace ρ clip	1.0		
		Optimizer	Adam		
		Seed	42		

B.4 SMART model training

We trained SMART models in CATK on subsets of the Waymo Open Motion Dataset (WOMD). WOMD motion shards were preprocessed into CATK’s per-scenario cached format, and all training subsets were built from these cached scenario files. Our final local runs used `smart_mini_3M` with vehicle-only supervision on deterministic subsets of 67, 200, 1200, and 12000 scenarios. In the subset construction scripts we used, scenarios are sorted by cached scenario filename in lexicographic order. Vehicle-only supervision means that only vehicle agents contributed to the training loss, while pedestrians and cyclists were still present in the scene and available as context to the model. The local models were trained with CATK’s `pre.bc` configuration on a single GPU for 64 epochs with batch size 8. These runs used standard supervised behavioral cloning and were not further fine-tuned with CAT-K / CLSFT. We additionally compare against two author-provided checkpoints: a behavioral-cloning checkpoint (`pre.bc.E31.ckpt`) and a closed-loop supervised fine-tuned checkpoint (`clsft.E9.ckpt`). For downstream evaluation, we exported predictions as `.pkl` files on the 10k validation scenes from the `pufferdrive_womd_subsets` dataset. We used two export modes: an all-agents mode and a planning mode in which only the SDC is controlled by the model and all other agents are replayed from ground truth. We re-exported all model and export-mode combinations with 32 rollouts for

multimodal evaluation. We verified that the scenario-ID overlap between each local training subset (67, 200, 1200, and 12000) and the evaluation set is zero.

C Neural network architecture

Both the BC anchor and the RL policy share the same multi-modal encoder structure. The flattened environment observation vector is first unpacked into its modalities: ego state, partner agents, and road segments. Each modality is processed by a two-layer MLP with ReLU activation and layer normalisation between the two linear layers. Partner and road embeddings are then aggregated across objects via max-pooling, producing one vector per stream. The three pooled vectors are concatenated and passed through a shared two-layer MLP (Linear $\rightarrow$ ReLU $\rightarrow$ Linear) to produce the final embedding. Separate linear heads decode this embedding into logits over each action dimension; a separate linear head with unit output produces the value estimate. The two architectures differ in width and in the presence of recurrence:

- **BC anchor.** Per-stream MLP width 128, shared MLP $3 \times 128 \rightarrow 512 \rightarrow 512$. No recurrence. Actor heads are linear projections from the 512-dimensional embedding. Total: 776,190 parameters.
- **RL policy.** Per-stream MLP width 64, shared MLP $3 \times 64 \rightarrow 256 \rightarrow 256$. The 256-dimensional embedding is passed through a single-layer LSTM with input size 256 and hidden size 256 (PufferLib LSTMWrapper). Actor and critic heads are linear projections from the 256-dimensional LSTM output. Total: **TODO**.

Road segment features include a categorical type field that is replaced by a 7-class one-hot vector before encoding, expanding the road feature dimension from d_{road} to $d_{\text{road}} + 6$.

D Evaluation

D.1 Filtering the Waymo dataset for interactive SDC scenarios

Most scenarios in the Waymo Open Motion Dataset (WOMD) involve the self-driving car (SDC) traveling without meaningful interaction with other agents—the SDC reaches its destination without requiring coordination or yielding. To increase the signal in our human-replay evaluation, we filter the dataset for scenarios in which the SDC trajectory intersects with other agents’ trajectories, indicating situations that require coordination, such as merging, yielding, or navigating busy intersections.

We score each scenario by counting the number of segment-level intersections between the SDC trajectory and all other agent trajectories, optionally filtering crossings that meet a minimum acute-angle threshold (to exclude near-parallel overlaps, such as lane changes). From a pool of 10,000 held-out validation scenarios, we rank by intersection count and select the top 200 most interactive scenes. Figure 7 shows the resulting intersection count distributions across the full dataset and the selected subset, and Figure 8 shows nine representative examples from the selected set.

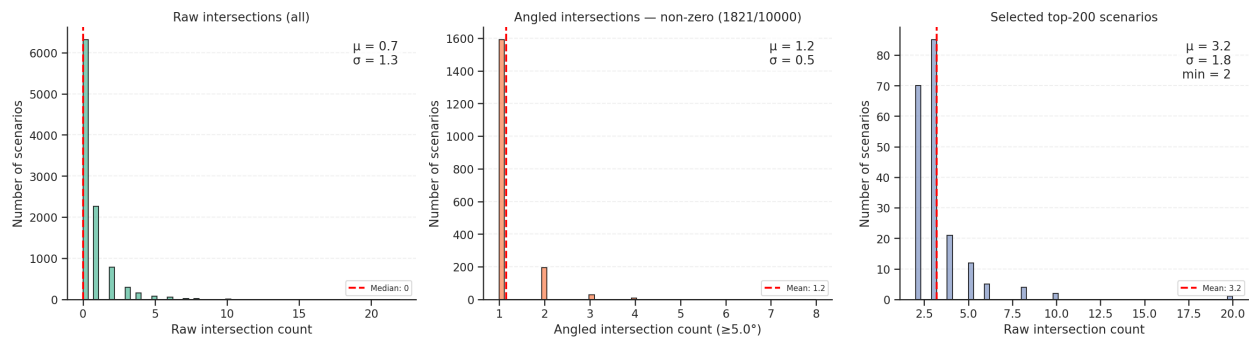


Figure 7: Distribution of SDC trajectory intersection counts. **Left:** raw intersection counts across all 50k scenarios. **Center:** angled intersections (non-zero only). **Right:** distribution within the selected top-200 subset.

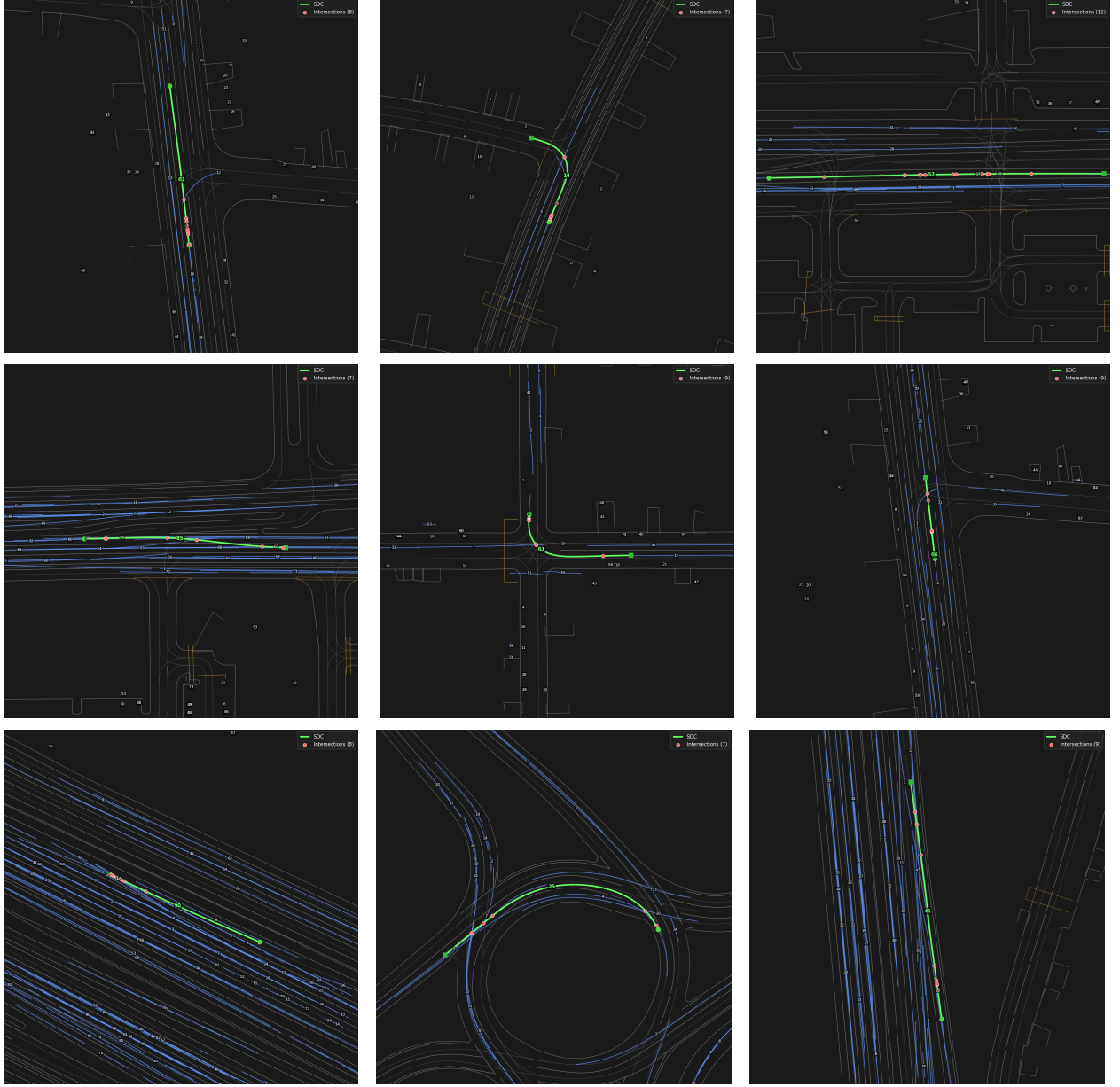


Figure 8: Nine example scenarios from the selected interactive subset. The SDC trajectory is shown in green, other agents in blue, and trajectory intersection points with other logs in red.

D.2 Metrics

We report the following metrics across all experiments. Unless noted, all metrics are computed per active agent per episode and averaged across agents and scenarios.

Score. The primary metric. An agent scores 1 if it reaches its goal without any collision or off-road event during the episode, and 0 otherwise. It jointly captures all failure modes and is the main basis for comparing conditions.

Completion rate. The fraction of agents that reach their goal position (within $\delta = 2$ meters) before episode end, regardless of whether a collision or off-road event occurred.

Collision rate. The fraction of episodes in which the agent is involved in at least one collision with another vehicle.

At-fault collision rate. A subset of the collision rate taken from NAVSIM [13]. A collision is attributed to an agent if (i) the other vehicle is in front of the agent at the time of impact, and (ii) the agent’s velocity vector points toward the other vehicle. This filters out collisions in which the agent was rear-ended or struck laterally by an inattentive partner.

Rear collision rate. The complement of at-fault: a collision is flagged as a rear collision for agent i if the colliding partner satisfies the at-fault conditions relative to i . A high rear collision rate indicates that the agent is being struck from behind, which is a signal that it is not keeping pace with traffic flow.

Off-road rate. The fraction of episodes in which the agent crosses a road edge boundary, detected by checking for intersection between the agent bounding box and any road edge polyline.

Route progress ratio. Following [17], we measure how far along its expert reference trajectory each agent travels. At each timestep t , we find the closest point $x(t)$ on the agent’s logged trajectory and compute its arc-length distance $d_{x(t)}$ from the start of the path. The route progress ratio is

$$\rho = \frac{d_{x(t)} - d_p}{d_q - d_p}, \quad (20)$$

where d_p and d_q are the arc-length distances to the initial and final positions of the logged trajectory, respectively. A value of $\rho = 1$ means the agent reached its destination; $\rho > 1$ is possible if the agent overshoots. For agents that reach their goal under GOAL_REMOVE termination, we set $\rho = 1$ directly, since their position is invalidated upon removal. For all other agents, ρ is computed from the agent’s final position at episode end.

Lateral deviation. At each timestep t , we compute the Euclidean distance from the agent’s current position to the nearest point on its expert reference trajectory. We average this distance over all timesteps for which the agent is alive, yielding a mean lateral deviation in meters. This measures how far the agent drifts sideways from its intended route; a perfect replay would achieve zero lateral deviation.

ZSC gap. The difference in score between the self-play and human-replay evaluation conditions:

$$\Delta_{\text{ZSC}} = \text{score}_{\text{self-play}} - \text{score}_{\text{human-replay}}. \quad (21)$$

A value of zero indicates that the policy transfers without degradation to the human population.

E Additional results

@Daphne: TODO

E.1 Convergence and performance on the training dataset

Every policy is trained for 5 billion steps. Below, we plot the performance on the full training dataset as a sanity check for convergence. **TODO: In the plot below, you can clearly see that the regularized policies are not fully converged yet. My intuition is that once this is fixed, the red perf will be much better as well.**

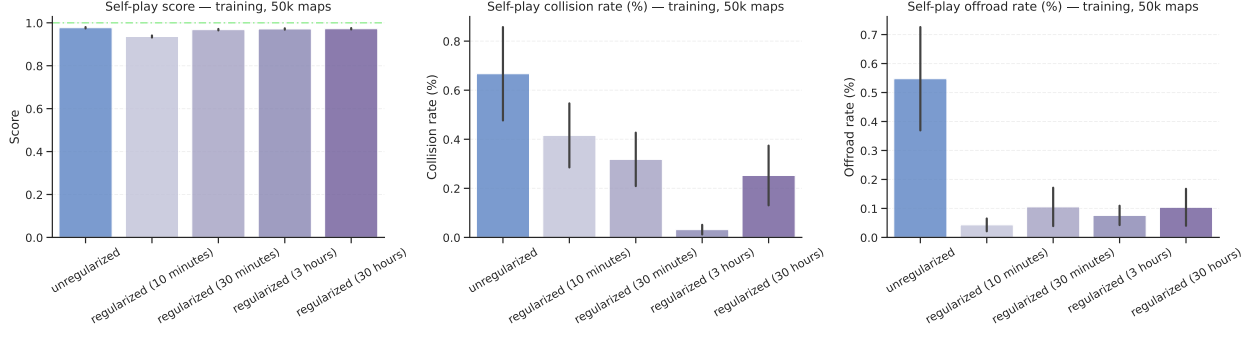


Figure 9: Performance of the policies trained on 50k maps on the full train dataset.

E.2 Human-replay results comparison

Table 8: Human-replay performance on the randomly sampled validation set vs. the interactive validation subset for all scaling checkpoints. Top-3 values per column are highlighted (best, 2nd, 3rd); best value additionally in bold.

Self-play maps (metadata)	Anchor data (human demonstrations)	HR — validation (random)						HR — interactive					
		Score ↑	Coll. (%) ↓	At-fault (%) ↓	Rear coll. (%) ↓	Route prog. ↑	Lateral L2 ↓	Score ↑	Coll. (%) ↓	At-fault (%) ↓	Rear coll. (%) ↓	Route prog. ↑	Lateral L2 ↓
10	0 (unreg.)	0.637 ± 0.007	23.3 ± 0.6	21.0 ± 0.6	12.9 ± 0.5	0.989 ± 0.001	3.34 ± 0.11	0.378 ± 0.007	47.1 ± 0.7	43.8 ± 0.7	33.3 ± 0.7	0.990 ± 0.001	5.85 ± 0.15
100	0 (unreg.)	0.784 ± 0.006	15.7 ± 0.5	13.4 ± 0.5	7.9 ± 0.4	0.992 ± 0.001	2.50 ± 0.10	0.620 ± 0.007	31.9 ± 0.7	26.7 ± 0.6	20.1 ± 0.6	0.989 ± 0.001	5.27 ± 0.15
1k	0 (unreg.)	0.925 ± 0.004	5.1 ± 0.3	3.5 ± 0.3	2.5 ± 0.2	0.997 ± 0.001	1.40 ± 0.06	0.848 ± 0.005	8.9 ± 0.4	4.1 ± 0.3	6.0 ± 0.3	0.982 ± 0.002	3.69 ± 0.14
10k	0 (unreg.)	0.963 ± 0.003	1.9 ± 0.2	1.1 ± 0.1	1.0 ± 0.1	0.997 ± 0.001	1.21 ± 0.05	0.900 ± 0.004	6.0 ± 0.3	2.7 ± 0.2	5.2 ± 0.3	0.994 ± 0.001	2.72 ± 0.09
50k	0 (unreg.)	0.962 ± 0.003	1.9 ± 0.2	1.1 ± 0.1	1.2 ± 0.2	0.999 ± 0.000	1.07 ± 0.04	0.900 ± 0.004	6.9 ± 0.4	2.8 ± 0.2	6.0 ± 0.3	0.998 ± 0.000	2.03 ± 0.06
10	10 minutes	0.716 ± 0.006	15.1 ± 0.5	11.6 ± 0.4	8.0 ± 0.4	0.964 ± 0.002	0.87 ± 0.02	0.513 ± 0.007	30.2 ± 0.6	21.0 ± 0.6	21.9 ± 0.6	0.960 ± 0.003	1.58 ± 0.02
10	30 minutes	0.729 ± 0.006	14.4 ± 0.5	10.8 ± 0.4	8.0 ± 0.4	0.980 ± 0.002	1.33 ± 0.04	0.477 ± 0.007	34.1 ± 0.7	28.9 ± 0.6	21.1 ± 0.6	0.965 ± 0.002	2.22 ± 0.04
10	3 hours	0.738 ± 0.006	14.1 ± 0.5	11.3 ± 0.4	6.2 ± 0.3	0.977 ± 0.002	1.33 ± 0.05	0.464 ± 0.007	27.8 ± 0.6	22.0 ± 0.6	16.7 ± 0.5	0.948 ± 0.002	3.42 ± 0.09
10	30 hours	0.769 ± 0.006	11.7 ± 0.4	9.0 ± 0.4	6.2 ± 0.3	0.981 ± 0.002	1.01 ± 0.02	0.501 ± 0.007	32.2 ± 0.7	26.6 ± 0.6	19.8 ± 0.6	0.979 ± 0.002	1.66 ± 0.02
100	10 minutes	0.867 ± 0.005	9.3 ± 0.4	7.4 ± 0.4	3.8 ± 0.3	0.997 ± 0.000	0.73 ± 0.01	0.753 ± 0.006	18.8 ± 0.5	13.6 ± 0.5	9.3 ± 0.4	0.989 ± 0.001	1.46 ± 0.02
100	30 minutes	0.879 ± 0.005	8.1 ± 0.4	6.2 ± 0.3	2.7 ± 0.2	0.996 ± 0.001	0.72 ± 0.01	0.772 ± 0.006	16.8 ± 0.5	13.7 ± 0.5	4.9 ± 0.3	0.984 ± 0.001	1.20 ± 0.01
100	3 hours	0.907 ± 0.004	6.0 ± 0.3	4.4 ± 0.3	2.2 ± 0.2	0.995 ± 0.001	0.71 ± 0.01	0.797 ± 0.006	14.3 ± 0.5	7.8 ± 0.4	11.1 ± 0.4	0.989 ± 0.001	1.47 ± 0.02
100	30 hours	0.906 ± 0.004	6.9 ± 0.4	4.8 ± 0.3	2.2 ± 0.2	0.994 ± 0.001	0.81 ± 0.01	0.808 ± 0.006	14.8 ± 0.5	9.9 ± 0.4	9.3 ± 0.4	0.984 ± 0.002	1.51 ± 0.02
1k	10 minutes	0.956 ± 0.003	2.4 ± 0.2	1.0 ± 0.1	1.8 ± 0.2	0.997 ± 0.000	0.68 ± 0.01	0.920 ± 0.004	5.9 ± 0.3	2.2 ± 0.2	3.9 ± 0.3	0.992 ± 0.001	1.32 ± 0.02
1k	30 minutes	0.962 ± 0.003	2.6 ± 0.2	0.7 ± 0.1	2.1 ± 0.2	0.997 ± 0.000	0.68 ± 0.01	0.920 ± 0.004	7.5 ± 0.4	1.0 ± 0.1	7.1 ± 0.4	1.000	1.20 ± 0.01
1k	3 hours	0.968 ± 0.002	1.8 ± 0.2	0.7 ± 0.1	1.3 ± 0.2	0.997 ± 0.001	0.68 ± 0.01	0.930 ± 0.004	4.3 ± 0.3	0.5 ± 0.1	3.9 ± 0.3	0.992 ± 0.001	1.31 ± 0.02
1k	30 hours	0.959 ± 0.003	2.4 ± 0.2	1.0 ± 0.1	1.8 ± 0.2	0.997 ± 0.001	0.68 ± 0.01	0.900 ± 0.004	5.8 ± 0.3	2.1 ± 0.2	4.1 ± 0.3	0.985 ± 0.002	1.28 ± 0.02
10k	10 minutes	0.953 ± 0.003	1.5 ± 0.2	0.4 ± 0.1	1.2 ± 0.2	0.995 ± 0.001	0.74 ± 0.02	0.964 ± 0.003	2.3 ± 0.2	0.0	2.3 ± 0.2	0.997 ± 0.000	1.30 ± 0.02
10k	30 minutes	0.963 ± 0.003	1.5 ± 0.2	0.3 ± 0.1	1.3 ± 0.2	0.992 ± 0.001	0.67 ± 0.01	0.952 ± 0.003	3.5 ± 0.3	1.4 ± 0.2	3.5 ± 0.3	0.992 ± 0.001	1.19 ± 0.02
10k	3 hours	0.977 ± 0.002	1.2 ± 0.2	0.3 ± 0.1	1.0 ± 0.1	0.995 ± 0.001	0.78 ± 0.01	0.945 ± 0.003	3.5 ± 0.3	0.6 ± 0.1	2.5 ± 0.2	0.985 ± 0.002	1.45 ± 0.02
10k	30 hours	0.977 ± 0.002	1.3 ± 0.2	0.3 ± 0.1	1.2 ± 0.2	0.996 ± 0.001	0.68 ± 0.01	0.925 ± 0.004	3.5 ± 0.3	0.0	3.5 ± 0.3	0.985 ± 0.002	1.17 ± 0.01
50k	10 minutes	0.936 ± 0.003	1.6 ± 0.2	0.4 ± 0.1	1.3 ± 0.2	0.992 ± 0.001	0.77 ± 0.01	0.954 ± 0.003	1.1 ± 0.1	0.5 ± 0.1	1.1 ± 0.1	0.996 ± 0.000	1.41 ± 0.03
50k	30 minutes	0.974 ± 0.002	1.2 ± 0.1	0.2 ± 0.1	1.1 ± 0.1	0.994 ± 0.001	0.77 ± 0.01	0.936 ± 0.003	4.0 ± 0.3	1.5 ± 0.2	3.6 ± 0.3	0.991 ± 0.001	1.49 ± 0.02
50k	3 hours	0.980 ± 0.002	0.7 ± 0.1	0.2 ± 0.1	0.6 ± 0.1	0.997 ± 0.001	0.74 ± 0.01	0.927 ± 0.004	1.8 ± 0.2	0.0	1.8 ± 0.2	0.985 ± 0.001	1.32 ± 0.02
50k	30 hours	0.970 ± 0.002	1.1 ± 0.1	0.2 ± 0.1	1.0 ± 0.1	0.995 ± 0.001	0.70 ± 0.01	0.951 ± 0.003	1.9 ± 0.2	0.5 ± 0.1	1.5 ± 0.2	0.989 ± 0.001	1.24 ± 0.01

E.3 Ablations

We report the results on the following ablations.

Sensor noise

Lambda conditioning

Reward conditioning